

MaxUnpool2d#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.pooling.MaxUnpool2d.html>

Description:

Computes a partial inverse of MaxPool2d. MaxPool2d is not fully invertible, since the non-maximal values are lost. MaxUnpool2d takes in as input the output of MaxPool2d including the indices of the maximal values and computes a partial inverse in which all non-maximal values are set to zero. This operation may behave nondeterministically when the input indices has repeat values. See [pytorch/pytorch#80827](#) and [Reproducibility](#) for more information.

Function Signature

```
class torch.nn.modules.pooling.MaxUnpool2d(kernel_size,
stride=None, padding=0)[source]#
```

Parameters

Inputs:

Shape:

```
forward(input, indices, output_size=None)[source]#
```

Returns:

Tensor

Code Examples

```
>>> pool = nn.MaxPool2d(2, stride=2, return_indices=True)
>>> unpool = nn.MaxUnpool2d(2, stride=2)
>>> input = torch.tensor([[[[ 1., 2., 3., 4.],
                             [ 5., 6., 7., 8.],
                             [ 9., 10., 11., 12.],
                             [13., 14., 15., 16.]]]]])

>>> output, indices = pool(input)
>>> unpool(output, indices)
tensor([[[[ 0., 0., 0., 0.],
           [ 0., 6., 0., 8.],
           [ 0., 0., 0., 0.],
           [ 0., 14., 0., 16.]]]]])

>>> # Now using output_size to resolve an ambiguous size for the
inverse
>>> input = torch.tensor([[[[ 1., 2., 3., 4., 5.],
                             [ 6., 7., 8., 9., 10.],
                             [11., 12., 13., 14., 15.],
                             [16., 17., 18., 19., 20.]]]]])

>>> output, indices = pool(input)
>>> # This call will not work without specifying output_size
>>> unpool(output, indices, output_size=input.size())
tensor([[[[ 0., 0., 0., 0., 0.],
           [ 0., 7., 0., 9., 0.],
           [ 0., 0., 0., 0., 0.],
           [ 0., 17., 0., 19., 0.]]]]])
```

Notes & Warnings

NOTE

This operation may behave nondeterministically when the input indices has repeat values. See [pytorch/pytorch#80827](#) and [Reproducibility](#) for more information.

NOTE

MaxPool2d can map several input sizes to the same output sizes. Hence, the inversion process can get ambiguous. To accommodate this, you can provide the needed output size as an additional argument `output_size` in the forward call. See the [Inputs and Example](#) below.

Detailed Documentation

Documentation

Computes a partial inverse of MaxPool2d.

MaxPool2d is not fully invertible, since the non-maximal values are lost.

MaxUnpool2d takes in as input the output of MaxPool2d including the indices of the maximal values and computes a partial inverse in which all non-maximal values are set to zero.

This operation may behave nondeterministically when the input indices has repeat values. See [pytorch/pytorch#80827](#) and [Reproducibility](#) for more information.

MaxPool2d can map several input sizes to the same output sizes. Hence, the inversion process can get ambiguous. To accommodate this, you can provide the needed output size as an additional argument `output_size` in the forward call. See the Inputs and Example below.

`kernel_size` (int or tuple) – Size of the max pooling window.

`stride` (int or tuple) – Stride of the max pooling window. It is set to `kernel_size` by default.

`padding` (int or tuple) – Padding that was added to the input

`input`: the input Tensor to invert

`indices`: the indices given out by MaxPool2d

`output_size` (optional): the targeted output size

Input: (N, C, H_{in}, W_{in}) or (N, C, H_{in}, W_{in}) or (C, H_{in}, W_{in}) or (C, H_{in}, W_{in})

Output: (N, C, H_{out}, W_{out}) or (N, C, H_{out}, W_{out}) or (C, H_{out}, W_{out}) or (C, H_{out}, W_{out}) , where

or as given by `output_size` in the call operator

Runs the forward pass.